

1. Introduction

For Task 7.4H, I decided to continue improving my Robot Battle game from my previous work.

Instead of creating a completely new project, I used my existing Robot Battle game and improved the Power-Up system. The game already had three Power-Ups: Extra Life, Rapid Fire and Shield.

The main improvement was changing the structure of the Power-Up system by using an abstract class, inheritance and polymorphism.

My goal was to make the code more organised, easier to understand and easier to extend. I also tested the changes through actual gameplay.

This task helped me demonstrate the three main areas of 7.4H: **Syntax, Concept and Process.**

2. What I Improved

The original Robot Battle game already had Power-Ups. For this task, I did not create the Power-Ups from the beginning. Instead, I improved their structure.

I created an abstract PowerUp class and three subclasses:

PowerUp

|

|--- ExtraLifePowerUp

|

|--- RapidFirePowerUp

|

|--- ShieldPowerUp

The PowerUp class contains the common information and behaviour.

The three subclasses inherit from PowerUp and provide their own effects.

This design is useful because the Power-Ups have some common features, such as their position, collision and drawing, but each Power-Up also has a different effect.

3. OOP Concepts Applied

3.1 Abstraction

I used an abstract class called PowerUp.

- public abstract class PowerUp

The class provides the common structure for all Power-Ups.

For example, it contains an abstract ApplyTo() method:

```
public abstract void ApplyTo(  
    PlayerRobot player,  
    ref int lives);
```

Each Power-Up needs to implement this method differently.

This helped me understand how an abstract class can provide a common structure without directly creating an object from the base class.

3.2 Inheritance

I used inheritance to create the three Power-Up classes.

- public class ExtraLifePowerUp : PowerUp
- public class RapidFirePowerUp : PowerUp
- public class ShieldPowerUp : PowerUp

The subclasses can reuse the common functionality from PowerUp while adding their own behaviour.

For example, ExtraLifePowerUp increases the player's lives, while RapidFirePowerUp activates rapid shooting.

3.3 Polymorphism

Polymorphism was one of the main concepts I applied in this task.

In the Game class, I used:

- private List<PowerUp> _powerUps;

This means different Power-Up objects can be stored in the same list.

For example, the list can contain:

- ExtraLifePowerUp
- RapidFirePowerUp
- ShieldPowerUp

The game then handles them through the common PowerUp type.

When the player collects a Power-Up, the game calls:

- powerUp.ApplyTo(_player, ref _lives);

The correct implementation is used depending on the actual Power-Up object.

This helped me understand polymorphism in a real program instead of only learning it from simple examples.

3.4 Encapsulation

I also used encapsulation in the Power-Up class.

For example:

- `public double X { get; private set; }`
- `public double Y { get; private set; }`

Other classes can read these values, but they cannot directly change them.

This allows the Power-Up class to control its own data and behaviour.

4. Power-Up Behaviour

Each Power-Up has a different effect.

Extra Life

The Extra Life Power-Up increases the player's lives when the player has less than five lives.

```
if (lives < MaximumLives)
{
    lives++;
}
```

The maximum number of lives is five.

Rapid Fire

The Rapid Fire Power-Up calls:

- `player.ActivateRapidFire();`

This changes the player's shooting cooldown and allows the player to shoot faster for a limited time.

Shield

The Shield Power-Up calls:

- `player.ActivateShield();`

The player then receives a temporary shield. The game checks the shield state during collision handling.

Therefore, the three subclasses have different behaviours even though they share the same PowerUp base class.

5. Testing

After changing the Power-Up structure, I ran the game and tested each Power-Up through actual gameplay.

Test 1 – Extra Life

Expected result:

The player's lives should increase by one when the player has less than five lives.

Test:

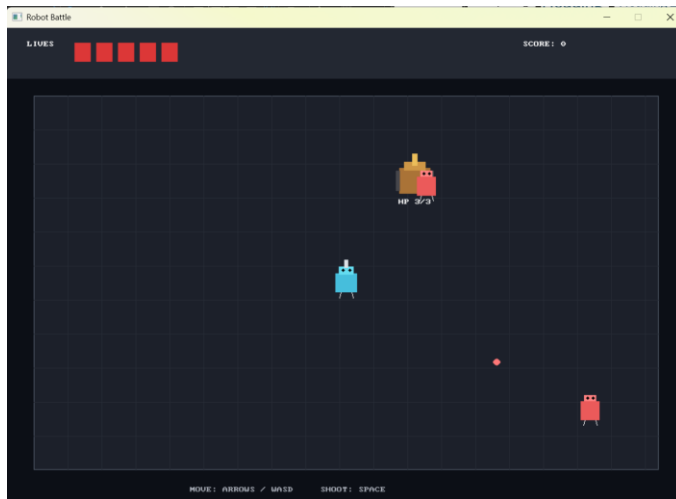
I first lost one life and then collected the L Power-Up.

Actual result:

The player's life increased successfully.

Result: Pass

Evidence:



Test 2 – Rapid Fire

Expected result:

The player should shoot faster after collecting the R Power-Up.

Test:

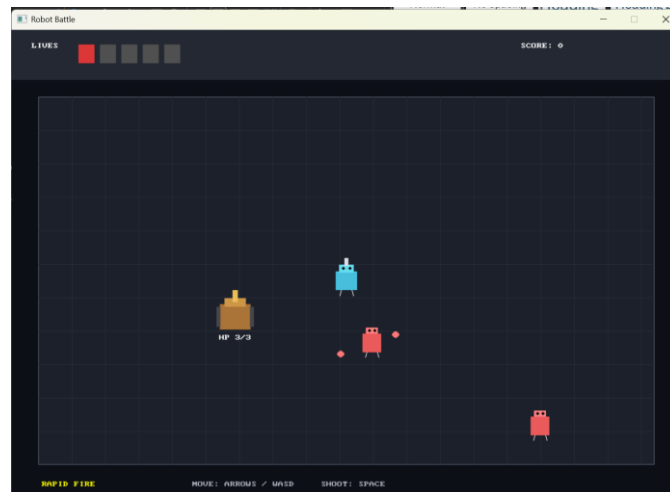
I collected the Rapid-Fire Power-Up during gameplay.

Actual result:

The RAPID-FIRE indicator appeared and the player's shooting speed increased.

Result: Pass

Evidence:



Test 3 – Shield

Expected result:

The player should receive a temporary shield after collecting the S Power-Up.

Test:

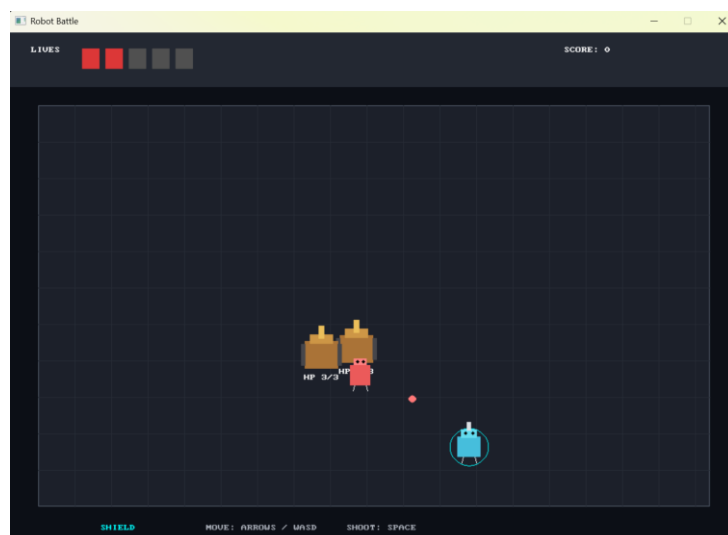
I collected the Shield Power-Up during gameplay.

Actual result:

The shield appeared around the player.

Result: Pass

Evidence:



Test 4 – Power-Up Removal

Expected result:

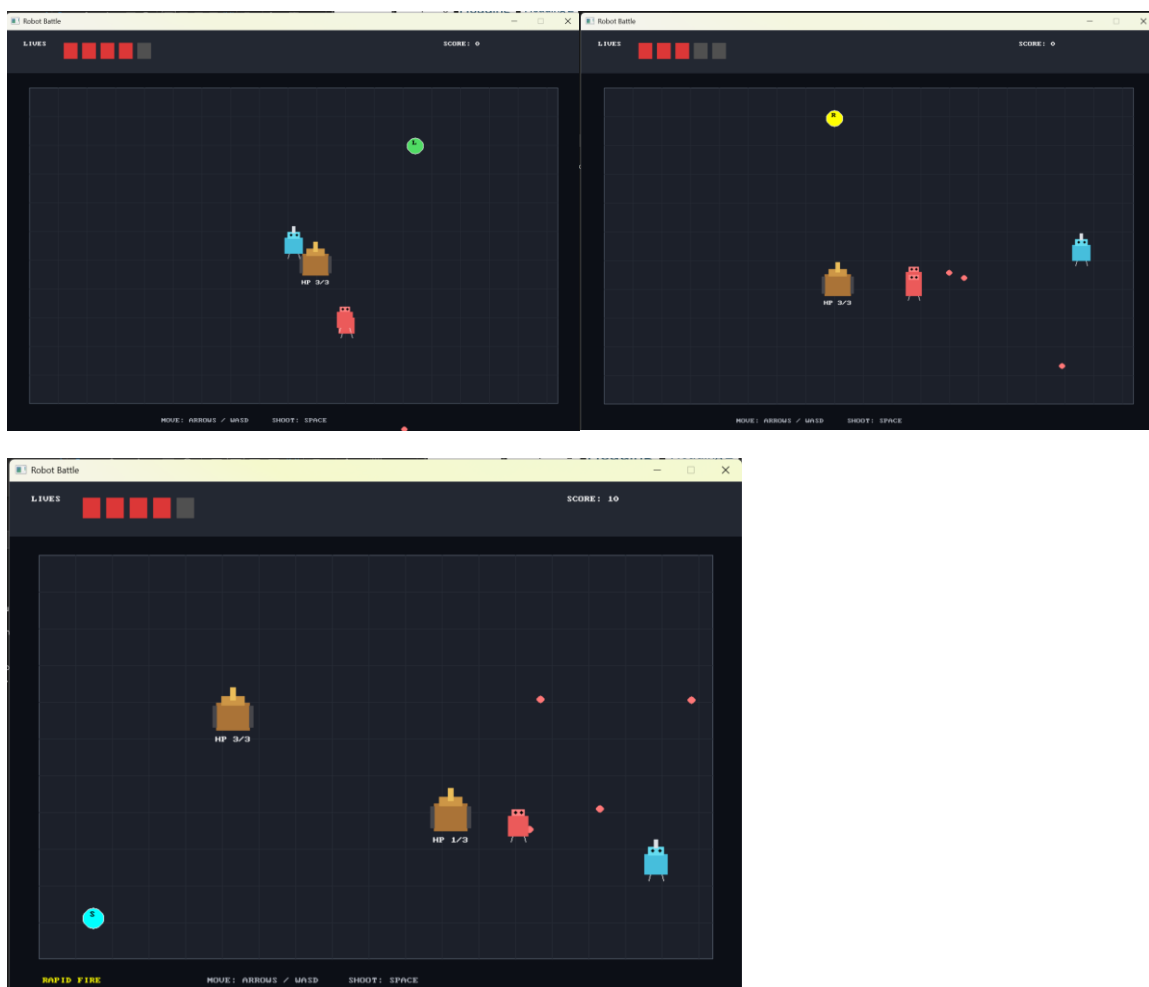
A Power-Up should disappear after it has been collected.

Actual result:

The collected Power-Up became inactive and was removed from the active Power-Up collection.

Result: Pass

The game sets the collected Power-Up to inactive and later removes inactive Power-Ups from the list.



6. Development Process

My development process was based on improving the existing code step by step.

First, I looked at the existing Power-Up system and identified that the Power-Ups had different behaviours.

Next, I created the abstract PowerUp class and moved the common functionality into it.

After that, I created:

- ExtraLifePowerUp
- RapidFirePowerUp
- ShieldPowerUp

I then changed the Game class to store the different Power-Ups using:

- List<PowerUp>

After implementing the changes, I ran the game and tested each Power-Up.

Finally, I checked that the existing Power-Up functions still worked correctly.

This process helped me improve the code without changing the main gameplay.

7. Syntax, Concept and Process

Syntax

For this task, I improved my use of C# syntax and coding conventions.

I used:

- abstract classes
- inheritance
- override
- constructors
- properties
- access modifiers
- private fields
- constants
- methods
- List<T>

I also used meaningful names such as PowerUp, ExtraLifePowerUp, RapidFirePowerUp and ShieldPowerUp.

I used constants for values such as the Power-Up duration and shooting cooldown. This makes the code easier to understand and maintain.

This demonstrates the Syntax part of the task because I focused on clean and organised C# code.

Concept

The main concept I focused on was applying OOP concepts to a real game.

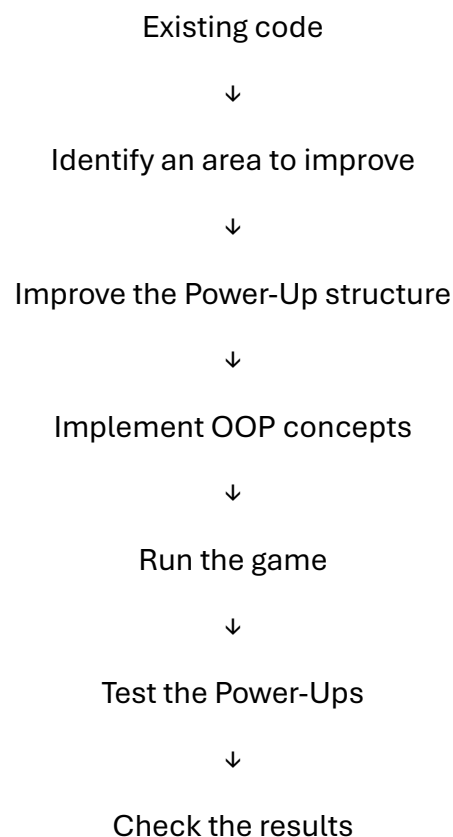
I used abstraction to create the common Power-Up structure, inheritance to create the different Power-Ups, polymorphism to manage them through List<PowerUp>, and encapsulation to protect object data.

This helped me understand how these OOP concepts work together in a larger program.

Process

I did not only change the code and assume it worked.

I followed a simple process:

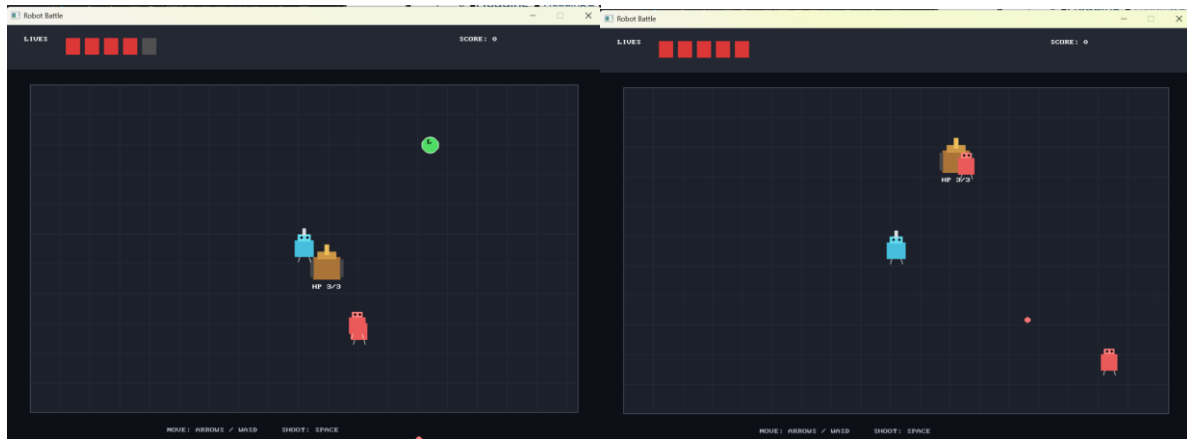


I used actual gameplay to check the final implementation. This demonstrates the Process part of the task because I refined the existing code and tested the result.

8. Evidence

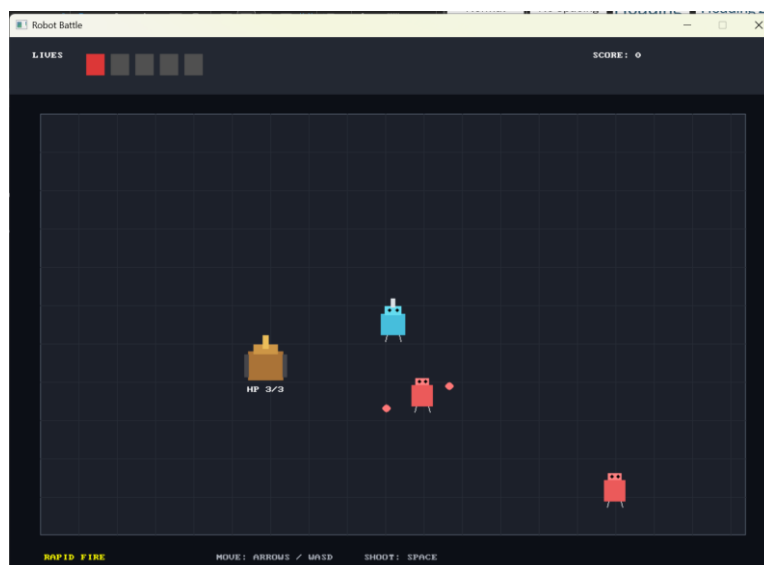
The screenshots below provide evidence of my implementation and testing.

Figure 1 – Extra Life Power-Up



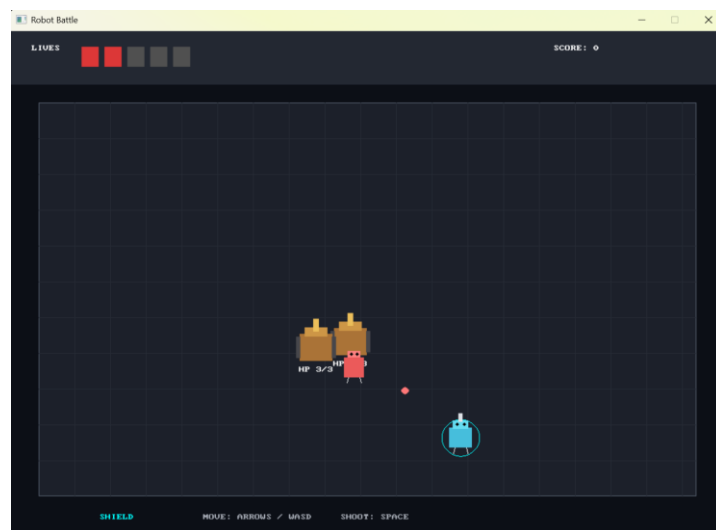
The screenshot shows the Extra Life Power-Up being collected after the player had lost a life.

Figure 2 – Rapid Fire Power-Up



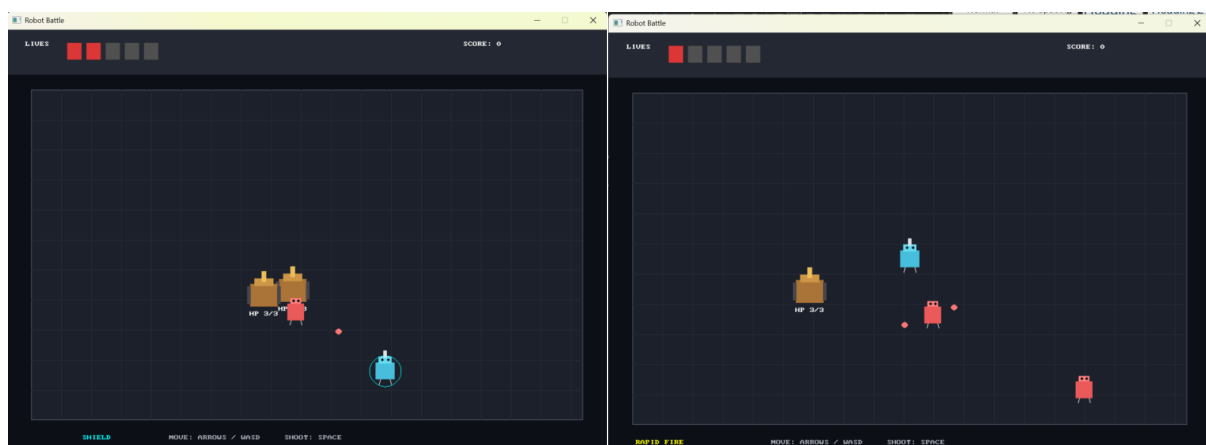
The screenshot shows the Rapid-Fire effect being activated.

Figure 3 – Shield Power-Up



The screenshot shows the Shield effect around the player.

Figure 4 – Power-Up System



The screenshot shows the Power-Up system operating as part of the Robot Battle game.

9. Unit Learning Outcomes

This task helped me apply the OOP concepts I learned throughout the unit to a working C# game.

I demonstrated abstraction by creating the PowerUp abstract class, inheritance through the three Power-Up subclasses, and polymorphism by using List<PowerUp>.

I also used encapsulation to control the Power-Up data and tested the implementation through actual gameplay.

This shows how I can apply OOP concepts to a practical programming problem instead of only explaining them in theory.

10. Reflection

One of the main things I learned from this task was that improving a program is not always about adding a completely new feature.

My Robot Battle game already had Power-Ups, but I realised that the structure could be improved.

By creating an abstract PowerUp class and different subclasses, I could separate the different Power-Up behaviours while keeping the common parts together.

I also gained a better understanding of polymorphism. The Game class uses List<PowerUp>, while the actual objects can be different Power-Up subclasses.

Testing was also important. I tested Extra Life, Rapid Fire and Shield separately and confirmed that they continued to work after the changes.

Overall, this task helped me become more confident in applying OOP concepts to a larger program. It also showed me that good programming is not only about making the program run, but also about making the code organised and easier to extend.

11. Related Material

GitHub Repository:

<https://github.com/sj226384658/SIT771.git>

The GitHub repository contains the Robot Battle source code related to this task.

The 7.4H submission requires a document explaining the work and a link to the related material.

12. Conclusion

For Task 7.4H, I improved the existing Robot Battle Power-Up system by using abstraction, inheritance, polymorphism and encapsulation.

The original Power-Up functionality was maintained, while the internal structure was improved.

I also tested Extra Life, Rapid Fire and Shield through actual gameplay, and all three worked correctly.

This task helped me apply the OOP knowledge I developed throughout the unit to a practical C# game and demonstrate **Syntax, Concept and Process**.